PYTHON PROGRAMMING

Learn • Practice • Build

Begin your programming journey with one of the world's most powerful and beginner-friendly languages.

INSTRUCTOR EDITION

Mohanraj R

Python Trainer | AI & ML Enthusiast

mohanraj9677011@gmail.com

linkedin.com/in/moganraj

github.com/lookmohan

For College Entry Level Students | June 2026

TABLE OF CONTENTS

01 Python Basics — Data Types, Variables & Features

02 Type Casting

03 User Input

04 Operators

05 Conditional Statements

06 For Loop and Nested Loops

07 While Loop and Loop Control (break/continue/pass)

08 Functions and Lambda

09 Modules and Packages

10 String Manipulation

11 Lists

12 Tuples

13 Sets

14 Dictionaries

15 Object-Oriented Programming (OOP)

16 File Handling

17 Exception Handling

18 Iterators, Generators and Decorators

19 Logging and Collections

20 Mini Projects

Chapter 01 — Python Basics

Data Types, Variables and Python Features

What is Python?

Python is a high-level, interpreted, open-source programming language. It is easy to read, beginner-friendly, and widely used in web development, data science, AI, and automation.

Python Features

- Open source and free to use
- Easy to read and understand
- High-level programming language
- Interpreted language — executes line by line
- Supports Object-Oriented Programming (OOP)

Data Types

A data type tells Python what kind of value a variable holds.

Data Type	Keyword	Example
Integer	`int`	`1, 2, 100`
String	`str`	`'hello', "world"`
Float	`float`	`3.14, 1.0`
Boolean	`bool`	`True, False`

Variables

A variable is a container that stores a value.

Variable Naming Rules

- Can contain letters, digits, and underscore (_)
- Cannot start with a digit e.g. 1name is INVALID
- Variable names are case-sensitive: name and Name are different
- Avoid Python keywords like if, else, for, while, etc.

Valid vs Invalid Examples

```
# Valid variable names
first_name = "Ravi"
name1 = "Raj"
_score = 95

# Invalid variable names
# 1name = "error" <- starts with a digit
# my-name = "error" <- hyphens not allowed
```

Comments

Comments are lines Python ignores. They help explain code.

```
# This is a single-line comment

"""
This is a
multi-line comment
"""
```

Chapter 02 — Type Casting

Converting one data type into another

Type casting means converting a value from one data type to another. Python allows this because different operations require different data types.

> *Note: User input always comes as a string. To do calculations, you must convert it to int or float.*

Types of Type Casting

1. Implicit Type Casting

Python automatically converts data types without any instruction from the programmer.

```
a = 10 # int
b = 10.1 # float
c = a + b # Python auto-converts int to float
print(c) # Output: 20.1
print(type(c)) # Output: float
```

2. Explicit Type Casting

We manually convert data types using built-in functions: int(), float(), str(), bool()

```
# String to Integer
a = "100"
b = int(a)
print(b) # Output: 100
print(type(b)) # Output: int

# Integer to Float
a = 10
b = float(a)
print(b) # Output: 10.0

# Float to Integer (decimal part is cut off, NOT rounded)
a = 10.9
b = int(a)
print(b) # Output: 10

# Integer to String
a = 50
b = str(a)
print(b) # Output: '50'
```

> *Note: int() removes the decimal part. It does NOT round the number. 10.9 becomes 10.*

Chapter 03 — User Input

Taking values from the user while the program runs

The input() function pauses the program and waits for the user to type something. Whatever the user types is ALWAYS returned as a string.

```
name = input("Enter your name: ")
print("Hello,", name)
```

Problem Without Type Casting

Since input() returns a string, adding two numbers without casting gives wrong results:

```
# WRONG — this joins strings, not adds numbers
a = input("Enter first number: ")
b = input("Enter second number: ")
print(a + b) # User types 5 and 3 -> Output: 53 (not 8!)
```

Correct Approach — With Type Casting

```
# CORRECT — convert to int first
a = int(input("Enter first number: "))
b = int(input("Enter second number: "))
print(a + b) # Output: 8
```

Practice Program — Student Marks

```
name = input("Enter student name: ")
mark1 = int(input("Enter mark 1: "))
mark2 = int(input("Enter mark 2: "))
total = mark1 + mark2
average = total / 2
print("Name :", name)
print("Total :", total)
print("Average :", average)
```

Chapter 04 — Operators

Symbols used to perform operations on values

1. Arithmetic Operators

```
print(10 + 3) # Addition -> 13
print(10 - 3) # Subtraction -> 7
print(10 * 3) # Multiplication -> 30
print(10 / 3) # Division -> 3.333...
print(10 // 3) # Floor Division -> 3
print(10 % 3) # Modulus -> 1 (remainder)
print(2 ** 3) # Exponent -> 8 (2 to the power 3)
```

2. Assignment Operators

```
x = 10
x += 5 # x = x + 5 -> 15
x -= 3 # x = x - 3 -> 12
x *= 2 # x = x * 2 -> 24
x //= 4 # x = x // 4 -> 6
```

3. Comparison Operators

Always returns True or False.

```
print(5 == 5) # Equal -> True
print(5 != 3) # Not equal -> True
print(5 > 3) # Greater than -> True
print(5 < 3) # Less than -> False
print(5 >= 5) # Greater or equal -> True
print(5 <= 4) # Less or equal -> False
```

4. Logical Operators

```
print(True and True) # Both must be True -> True
print(True and False) # One is False -> False
print(True or False) # At least one True -> True
print(not True) # Reverses result -> False
```

5. Identity Operators

```
a = [1, 2, 3]
b = a
c = [1, 2, 3]
print(a is b) # True — same object in memory
print(a is c) # False — different objects (same content)
print(a is not c) # True
```

6. Membership Operators

```
fruits = ["apple", "mango", "grape"]
print("mango" in fruits) # True
print("banana" in fruits) # False
```

```
print("banana" not in fruits) # True
```

7. Bitwise Operators

Work on binary (0s and 1s) representation of numbers.

```
print(5 & 3) # AND -> 1
print(5 | 3) # OR -> 7
print(5 ^ 3) # XOR -> 6
print(~5) # NOT -> -6
print(5 << 1) # Left Shift -> 10 (multiply by 2)
print(5 >> 1) # Right Shift -> 2 (divide by 2)
```

Chapter 05 — Conditional Statements

Control your program's flow based on conditions

if Statement

```
age = 20
if age >= 18:
print("Eligible to vote")
```

if-else Statement

```
number = int(input("Enter a number: "))
if number % 2 == 0:
print("Even number")
else:
print("Odd number")
```

if-elif-else Statement

Used when you need to check multiple conditions one after another.

```
mark = int(input("Enter your mark: "))

if mark >= 90:
print("Grade A")
elif mark >= 75:
print("Grade B")
elif mark >= 50:
print("Grade C")
elif mark >= 35:
print("Grade D")
else:
print("Fail")
```

Nested if Statement

An if-else placed inside another if or else block.

```
age = int(input("Enter age: "))
has_id = input("Do you have an ID? (yes/no): ")

if age >= 18:
if has_id == "yes":
print("Access granted")
else:
print("ID required")
else:
print("Too young to enter")
```

Practice Questions

- 1. Check if a number is greater than 15
- 2. Check if a number is odd or even

- 3. Check if a number is positive or negative
- 4. Print student grade based on marks (A/B/C/D/Fail)

Chapter 06 — For Loop and Nested Loops

Repeat code a fixed number of times

A loop repeats code multiple times, saving time and reducing repetition.

For Loop Syntax

```
for variable in sequence:
statement
```

range() Function

```
# range(stop) – starts from 0
for i in range(5): # 0, 1, 2, 3, 4
print(i)

# range(start, stop)
for i in range(1, 6): # 1, 2, 3, 4, 5
print(i)

# range(start, stop, step)
for i in range(1, 20, 2): # 1, 3, 5, 7 ... 19
print(i)

# Reverse
for i in range(10, 0, -1): # 10, 9, 8 ... 1
print(i)
```

Examples

```
# Multiplication table of 5
for i in range(1, 11):
print("5 x", i, "=", 5 * i)

# Sum from 1 to 10
total = 0
for i in range(1, 11):
total += i
print("Sum:", total) # 55

# Count vowels in a word
word = "apple"
count = 0
for char in word:
if char in "aeiou":
count += 1
print("Vowels:", count) # 2
```

Nested For Loop

A loop inside another loop. The inner loop runs completely for each outer loop value.

```
# Pattern: numbers
for i in range(1, 4):
```

```
for j in range(1, 4):
print(i, end=" ")
print()
# Output:
# 1 1 1
# 2 2 2
# 3 3 3

# Triangle pattern
for i in range(1, 6):
for j in range(i):
print("*", end=" ")
print()
# Output:
# *
# * *
# * * *
# * * * *
# * * * * *

# Reverse triangle
for i in range(6, 0, -1):
for j in range(i):
print("*", end=" ")
print()
```

Practice Questions

- 1. Print numbers from 1 to 5
- 2. Print all even numbers from 1 to 20
- 3. Print all odd numbers from 1 to 20
- 4. Print multiplication table of a number entered by the user
- 5. Print your name four times using a loop
- 6. Find the sum of numbers from 1 to 100
- 7. Print square values from 1 to 10 (1, 4, 9, 16 ...)
- 8. Print a diamond pattern using nested loops

Chapter 07 — While Loop and Loop Control

Run code while a condition is True

The while loop runs as long as a condition is True. Use it when you do not know exactly how many times to repeat.

```
count = 1
while count <= 5:
print("Count:", count)
count += 1 # Always update — or you get an infinite loop!

# Output: Count: 1 Count: 2 Count: 3 Count: 4 Count: 5
```

User Input with While Loop

```
while True:
name = input("Enter name (or 'quit' to stop): ")
if name == "quit":
break
print("Hello,", name)
```

break — Stop the Loop Completely

```
for i in range(10):
if i == 5:
break # loop stops here
print(i)
# Output: 0 1 2 3 4
```

continue — Skip Current Iteration

```
for i in range(6):
if i == 3:
continue # skip 3, go to next
print(i)
# Output: 0 1 2 4 5
```

pass — Placeholder (Do Nothing)

```
for i in range(5):
if i == 3:
pass # does nothing, continues normally
print(i)
# Output: 0 1 2 3 4
```

Note: Use pass when you want to write code later but need the block structure now.

Chapter 08 — Functions and Lambda

Write reusable blocks of code

A function is a block of reusable code. Instead of writing the same code again and again, we create a function and call it whenever needed.

```
# Define a function
def greet():
print("Hello, Welcome!")

# Call the function
greet()
```

Types Based on Arguments and Return Value

```
# 1. No arguments, No return
def greet():
print("Hello!")
greet()

# 2. With arguments, No return
def add(a, b):
print(a + b)
add(10, 20) # Output: 30

# 3. No arguments, With return
def get_pi():
return 3.14159
result = get_pi()
print(result)

# 4. With arguments and return value (CORRECTED from notes)
def multiply(a, b):
return a * b # 'return' was misspelled in notes
result = multiply(4, 5)
print(result) # Output: 20
```

Types of Arguments

```
# 1. Positional Arguments — order matters
def student(name, age):
print(name, age)
student("Arun", 20)

# 2. Keyword Arguments — order does not matter
def employee(id, dept):
print("ID:", id)
print("Dept:", dept)
employee(dept="IT", id=101)

# 3. Default Arguments — used when argument is not passed
def city(place="Chennai"):
print("City:", place)
```

```
city() # Output: Chennai
city("Madurai") # Output: Madurai

# 4. Variable Length Arguments (*args)
def marks(*score):
print(score)
print("Total:", sum(score))
marks(85, 90, 78, 92)
```

Lambda Functions (Anonymous Functions)

A lambda is a small one-line function without a name. Used for quick operations.

```
# Syntax: lambda arguments : expression

# Regular function
def square(x):
return x * x

# Same as lambda
square = lambda x: x * x
print(square(5)) # Output: 25 (CORRECTED: was print(x(a)) in notes)

# Lambda with two arguments
add = lambda a, b: a + b
print(add(3, 7)) # Output: 10
```

Local vs Global Variables

```
# Local Variable — only inside the function
def student():
name = "Arun" # local
print(name)
student()
# print(name) <- This would give an error outside

# Global Variable — accessible anywhere
city = "Chennai" # global

def show():
print(city) # can use global variable

show()
print(city)
```

Chapter 09 — Modules and Packages

Organise and reuse code across files

A module is a Python file (.py) that contains functions, variables, and classes. Import it whenever needed instead of rewriting code.

Importing Modules

```
# 1. Normal import
import math
print(math.sqrt(25)) # 5.0
print(math.factorial(5)) # 120
print(math.pi) # 3.14159...

# 2. Import specific function
from math import sqrt
print(sqrt(16)) # 4.0

# 3. Import multiple functions
from math import sqrt, factorial
print(sqrt(9))
print(factorial(4))

# 4. Import all functions (use carefully)
from math import *
print(sqrt(49))

# 5. Alias (rename on import)
import math as m
print(m.sqrt(36))
```

Useful Built-in Modules

```
import random
print(random.randint(1, 10)) # random number 1 to 10
print(random.choice(["a", "b", "c"])) # random item from list

import datetime
now = datetime.datetime.now()
print(now) # current date and time

import os
print(os.getcwd()) # current working directory
```

Packages

A package is a folder containing multiple modules. It must have a file called __init__.py inside.

```
# Installing third-party packages using pip
# Run in terminal:
# pip install requests

import requests
response = requests.get("https://api.github.com")
```

```
print(response.status_code) # 200
```

Chapter 10 — String Manipulation

Working with text data in Python

A string is a sequence of characters written inside single or double quotes.

```
word = "python"
# Index: 0 1 2 3 4 5
# Neg idx: -6 -5 -4 -3 -2 -1

print(word[0]) # p
print(word[2]) # t
print(word[-1]) # n (last character)
```

Basic String Operations

```
# Concatenation
first = "Hello"
second = "World"
print(first + " " + second) # Hello World

# Repetition
print("Hi " * 3) # Hi Hi Hi

# Length
name = "python"
print(len(name)) # 6
```

String Slicing

```
# Syntax: string[start : stop : step]
name = "python"
print(name[2:4]) # th
print(name[:4]) # pyth
print(name[2:]) # thon
print(name[::2]) # pto (every 2nd character)
print(name[::-1]) # nohtyp (REVERSE the string!)
```

Useful String Methods

```
text = "hello world"
print(text.upper()) # HELLO WORLD
print(text.title()) # Hello World
print(text.replace("world", "Python")) # hello Python
print(text.split(" ")) # ['hello', 'world']
print(text.strip()) # removes leading/trailing spaces
print(text.startswith("he")) # True
print(text.count("l")) # 3

# f-string formatting (modern and recommended)
name = "Ravi"
age = 20
print(f"Name: {name}, Age: {age}")
```

Chapter 11 — Lists

Ordered, mutable collection of items

A list stores an ordered collection of items. It is mutable — you can change its contents.

```
numbers = [1, 2, 3, 4, 5]
mixed = [1, "hello", 3.14, True]
empty = []

fruits = ["apple", "mango", "grape"]
print(fruits[0]) # apple
print(fruits[-1]) # grape
```

List Operations

```
fruits = ["apple", "mango", "pineapple", "grape"]

# Add
fruits.append("banana") # add at end
fruits.insert(1, "watermelon") # add at index 1

# Remove
fruits.remove("mango") # remove by value
fruits.pop(0) # remove by index
fruits.pop() # remove last item

# Update
fruits[0] = "kiwi"

# Sort
numbers = [3, 1, 4, 1, 5, 9]
numbers.sort()
print(numbers) # [1, 1, 3, 4, 5, 9]
numbers.sort(reverse=True)
print(numbers) # [9, 5, 4, 3, 1, 1]

# Length and concatenation
print(len(fruits))
print([1,2,3] + [4,5,6]) # [1, 2, 3, 4, 5, 6]
```

List Slicing

```
numbers = [10, 20, 30, 40, 50, 60]
print(numbers[1:4]) # [20, 30, 40]
print(numbers[:3]) # [10, 20, 30]
print(numbers[3:]) # [40, 50, 60]
print(numbers[::-1]) # [60, 50, 40, 30, 20, 10]
```

Looping Through a List

```
fruits = ["apple", "mango", "grape"]

for item in fruits:
```

```
print(item)

for i, item in enumerate(fruits):
print(i, item)
```

Practice Questions

- 1. Create a list of 5 colours
- 2. Print the first and last item
- 3. Add one new item and remove one item
- 4. Print all items using a for loop
- 5. Find the length of the list
- 6. Sort the list and print it

Chapter 12 — Tuples

Ordered, immutable collection of items

A tuple is like a list but immutable — once created you cannot change, add, or remove items.

```
tuple1 = (1, 2, 3, 4, 5)
colors = ("red", "green", "blue")
single = (42,) # Single-item tuple needs a trailing comma!

fruits = ("apple", "mango", "pineapple", "grape")
print(fruits[0]) # apple
print(fruits[-1]) # grape
print(fruits[1:3]) # ('mango', 'pineapple')

print(len(fruits)) # 4

for item in fruits:
print(item)

a = (1, 2)
b = (3, 4)
print(a + b) # (1, 2, 3, 4)
```

List vs Tuple — Key Differences

Feature	List	Tuple
Mutable?	Yes	No
Symbol	`[ ]`	`( )`
Speed	Slower	Faster
Memory	More	Less
Use Case	Data that changes	Fixed/constant data

Practice Questions

- 1. Create a tuple with 5 numbers
- 2. Print the first and last item
- 3. Find the length of the tuple
- 4. Try changing an item — observe the error

Chapter 13 — Sets

Unordered collection of unique values

A set stores unique values only — duplicates are removed automatically. Sets are unordered, so items may appear in a different order each time.

```
fruits = {"apple", "mango", "grape"}
print(fruits)

# Duplicates removed automatically
numbers = {1, 2, 2, 3, 4, 4, 5}
print(numbers) # {1, 2, 3, 4, 5}

fruits.add("banana") # Add item
fruits.remove("mango") # Remove item

for item in fruits:
print(item)
```

Set Operations

```
a = {1, 2, 3, 4, 5}
b = {4, 5, 6, 7, 8}

# Union — combine both sets
print(a | b) # {1,2,3,4,5,6,7,8}
print(a.union(b))

# Intersection — common values only
print(a & b) # {4, 5}
print(a.intersection(b))

# Difference — values only in first set
print(a - b) # {1, 2, 3}
print(a.difference(b))

# Symmetric Difference — values NOT in both
print(a ^ b) # {1,2,3,6,7,8}
print(a.symmetric_difference(b))
```

Chapter 14 — Dictionaries

Store data as key: value pairs

A dictionary stores data in key:value pairs. Keys must be unique. You access values using their key, not an index.

```
student = {
"name" : "Dhanush",
"age" : 20,
"dept" : "CSE"
}

print(student["name"]) # Dhanush
print(student.get("age")) # 20
print(student.keys())
print(student.values())
print(student.items())
```

Modifying Dictionaries

```
student["blood_group"] = "B+" # Add new key
student["blood_group"] = "A+" # Update value
student.pop("dept") # Remove key
print(student)
```

Looping Through a Dictionary

```
student = {"name": "Ravi", "age": 21, "city": "Chennai"}

for key in student:
print(key)

for key, value in student.items():
print(key, ":", value)
```

All Data Structures — Quick Comparison

Type	Symbol	Duplicates	Mutable	Ordered
List	`[ ]`	Yes	Yes	Yes
Tuple	`( )`	Yes	No	Yes
Set	`{ }`	No	Yes	No
Dictionary	`{k:v}`	Keys: No	Yes	Yes (3.7+)

Chapter 15 — Object-Oriented Programming (OOP)

Classes, Objects, Inheritance, Polymorphism and Data Hiding

OOP organises code into classes and objects. It makes code reusable, structured, and easier to maintain.

Classes and Objects

```
class Student:
def __init__(self, name, age): # constructor
self.name = name
self.age = age

def display(self):
print("Name:", self.name)
print("Age :", self.age)

s1 = Student("Ravi", 20)
s2 = Student("Priya", 21)
s1.display()
s2.display()
```

Note: __init__ is the constructor — it runs automatically when you create an object.

Inheritance

A child class inherits all properties and methods of a parent class.

```
# Single Inheritance
class Animal:
def speak(self):
print("Animal speaks")

class Dog(Animal):
def bark(self):
print("Dog barks")

d = Dog()
d.speak() # Inherited from Animal
d.bark() # Dog's own method

# Multilevel Inheritance: Grandparent -> Parent -> Child
class Grandparent:
def a(self):
print("Grandparent")

class Parent(Grandparent):
def b(self):
print("Parent")

class Child(Parent):
def c(self):
print("Child")
```

```
obj = Child()
obj.a() # from Grandparent
obj.b() # from Parent
obj.c() # own
```

Polymorphism and Method Overriding

Same method name, different behaviour depending on which class calls it.

```
class Animal:
def speak(self):
print("Some sound")

class Dog(Animal):
def speak(self): # overrides parent method
print("Woof!")

class Cat(Animal):
def speak(self): # overrides parent method
print("Meow!")

animals = [Dog(), Cat(), Animal()]
for animal in animals:
animal.speak()
# Output: Woof! Meow! Some sound
```

Data Hiding (Encapsulation)

Use double underscore __ to make an attribute private — cannot be accessed from outside.

```
class Student:
def __init__(self):
self.__mark = 95 # private attribute

def show(self):
print("Mark:", self.__mark)

s = Student()
s.show() # Works fine -> Mark: 95
# print(s.__mark) # Error! Cannot access directly
```

Chapter 16 — File Handling

Create, read, write and manage files

File handling allows programs to permanently store data on disk, unlike variables which lose values when the program ends.

File Modes

Mode	Symbol	Description
Read	`r`	Open for reading (file must exist)
Write	`w`	Create new file or overwrite existing
Append	`a`	Add to end without deleting existing content
Read+Write	`r+`	Read and write (file must exist)

File Operations

```
# 1. Write to a file
file = open("notes.txt", "w")
file.write("Hello, Python!")
file.close()

# 2. Read from a file
file = open("notes.txt", "r")
print(file.read())
file.close()

# 3. Append data
file = open("notes.txt", "a")
file.write("\nNew line added")
file.close()

# 4. Read line by line
file = open("notes.txt", "r")
for line in file:
print(line, end="")
file.close()
```

Best Practice — using with open()

The with statement automatically closes the file when done.

```
# Read
with open("notes.txt", "r") as file:
data = file.read()
print(data)

# Write
with open("notes.txt", "w") as file:
file.write("Python is awesome!")
```

```
# Read and Write
with open("notes.txt", "r+") as file:
content = file.read()
print(content)
file.write("\nAdded via r+ mode")
```

Note: Always use 'with open()' in real projects — it is safer and cleaner.

Chapter 17 — Exception Handling

Handle errors so your program does not crash

An exception is an error that occurs during execution. Without handling it, Python stops the program and shows an error. Exception handling lets us control what happens when an error occurs.

Common Exceptions

Exception	Cause
ZeroDivisionError	Division by zero
ValueError	Invalid value e.g. int('abc')
TypeError	Wrong data type in operation
IndexError	List index out of range
KeyError	Dictionary key not found
FileNotFoundError	File does not exist
NameError	Variable used before being defined

try-except Syntax

```
try:
num = int(input("Enter a number: "))
print(10 / num)
except ZeroDivisionError:
print("Cannot divide by zero!")
except ValueError:
print("Please enter a valid number!")
except:
print("Some other error occurred")
```

try-except-finally

```
try:
file = open("data.txt", "r")
print(file.read())
except FileNotFoundError:
print("File not found!")
finally:
print("Program finished.") # always runs, error or not
```

Note: 'finally' always runs — use it to close files, database connections, etc.

Chapter 18 — Iterators, Generators and Decorators

Advanced Python concepts for cleaner, smarter code

Iterators

An iterator goes through items one by one. Every for loop uses an iterator internally. Needs two methods: __iter__() and __next__()

```
class MyNumbers:
def __iter__(self):
self.n = 1
return self

def __next__(self):
if self.n <= 5:
x = self.n
self.n += 1
return x
else:
raise StopIteration # stops the iterator

obj = MyNumbers()
for i in obj:
print(i) # Output: 1 2 3 4 5
```

Generators

A generator is an easy way to create an iterator using yield. yield pauses the function and remembers where it stopped.

```
def my_gen():
yield 1
yield 2
yield 3

for i in my_gen():
print(i) # Output: 1 2 3

# Generator for squares
def squares(n):
for i in range(1, n+1):
yield i * i

for val in squares(5):
print(val) # 1 4 9 16 25
```

Decorators

A decorator adds extra functionality to a function without changing the original function.

```
def wish(func):
def wrapper():
```

```
print("Good morning!")
func()
print("Have a nice day!")
return wrapper

@wish
def student():
print("I am a student.")

student()
# Output:
# Good morning!
# I am a student.
# Have a nice day!
```

Chapter 19 — Logging and Collections

Track program behaviour and use advanced data structures

Logging

Logging records what happens in your program. It is better than print() for real projects.

Level	When to Use
DEBUG	Detailed info for diagnosing problems
INFO	Confirm things are working as expected
WARNING	Something unexpected but not an error
ERROR	A serious problem occurred
CRITICAL	A very serious error — program may crash

```
import logging

logging.basicConfig(level=logging.DEBUG)

balance = 1000
logging.info("User logged in")
withdraw = 200

if withdraw > balance:
logging.error("Insufficient balance")
else:
balance -= withdraw
logging.info(f"Transaction successful. New balance: {balance}")
```

Collections Module

The collections module gives special data structures more useful than normal list/dict.

Counter — Count frequency of items

```
from collections import Counter

c = Counter(["a", "b", "b", "c", "c", "c"])
print(c) # Counter({'c': 3, 'b': 2, 'a': 1})

words = "apple banana apple cherry banana apple"
word_count = Counter(words.split())
print(word_count.most_common(2)) # Top 2 most common words
```

deque — Double-Ended Queue

```
from collections import deque

data = deque([1, 2, 3])
data.append(4) # add to right
```

```
data.appendleft(0) # add to left
print(data) # deque([0, 1, 2, 3, 4])

data.pop() # remove from right
data.popleft() # remove from left
print(data) # deque([1, 2, 3])
```

Note: deque is faster than a list for adding/removing items from both ends.

Chapter 20 — Mini Projects

Apply what you've learned — build real programs!

Project 01 — Simple Calculator

Build a calculator that performs addition, subtraction, multiplication, and division.

Skills Used: User Input, Type Casting, Operators, Conditional Statements

Step-by-Step Instructions:

1. Get two numbers from the user using input() and convert to float
2. Ask the user to choose an operation (+, -, *, /)
3. Use if-elif-else to perform the selected operation
4. Handle division by zero: if b == 0 print an error message
5. Display the result clearly
6. Bonus: Wrap in a while loop so the user can calculate multiple times

---

Project 02 — Number Guessing Game

The program picks a random number and the user must guess it.

Skills Used: random module, While Loop, Conditional Statements, Functions

Step-by-Step Instructions:

1. Import the random module
2. Generate a random number between 1 and 100 using random.randint()
3. Use a while loop to keep asking the user for a guess
4. Tell the user if their guess is Too High, Too Low, or Correct
5. Count and display how many attempts the user took
6. Bonus: Limit attempts to 10 and end the game if exceeded

Project 03 — Student Report Card

Take student name and marks for 5 subjects, calculate total, average, and grade.

Skills Used: Lists, Functions, Conditional Statements, Loops

Step-by-Step Instructions:

1. Create a function to collect marks for 5 subjects using a loop
2. Calculate total using sum() and average using total / 5
3. Use if-elif-else to assign a grade: A (90+), B (75+), C (50+), D (35+), Fail
4. Store student details in a dictionary
5. Display the full report card: name, marks, total, average, grade
6. Bonus: Store multiple students in a list of dictionaries

Project 04 — To-Do List App

A command-line app where users can add, view, and delete tasks.

Skills Used: Lists, While Loop, Functions, File Handling

Step-by-Step Instructions:

1. Create an empty list to store tasks
2. Show a menu: 1-Add Task 2-View Tasks 3-Delete Task 4-Quit
3. Use a while loop to keep showing the menu
4. Implement add, view, and delete as separate functions
5. Save tasks to a text file so they persist between program runs
6. Bonus: Add a 'Mark as Complete' feature

Project 05 — Contact Book

A simple app to store, search, and display contact information.

Skills Used: Dictionary, Lists, Functions, File Handling, Exception Handling

Step-by-Step Instructions:

1. Use a dictionary to store contacts (name as key, phone as value)
2. Create functions: add_contact(), view_contacts(), search_contact(), delete_contact()
3. Use a while loop with a menu to navigate between features
4. Handle 'contact not found' using try-except or if-else
5. Save contacts to a file and load them when the program starts
6. Bonus: Add email and address using nested dictionaries